



 Synthetics

Concepts

Synthetics

Synthetic instruments are locally defined instruments whose prices derive from other instruments. They can combine components from one venue or many venues and expose the result as a standard Nautilus instrument with the synthetic venue code `SYNTH`.

Synthetic instruments are useful for:

- Enabling `Actor` and `Strategy` components to subscribe to quote or trade feeds.
- Triggering emulated orders from derived prices.
- Constructing bars from synthetic quotes or trades.

Synthetic instruments cannot be traded directly. They exist locally within the platform and serve as analytical tools. In the future, Nautilus may support trading component instruments based on synthetic instrument behavior.

Formula language

Each synthetic instrument defines a derivation formula. Nautilus evaluates this formula with its built-in numeric expression engine and converts the final numeric result to the synthetic `Price`.

Supported syntax

Formulas can reference component `InstrumentId` values directly, including IDs that contain `/` and `-`.

Construct	Example	Notes
Component reference	<code>BTCUSDT.BINANCE</code>	Use the raw <code>InstrumentId</code> text.
Component reference	<code>AUD/USD.SIM</code>	IDs containing <code>/</code> are valid.
Component reference	<code>ETH-USDT-SWAP.OKX</code>	IDs containing <code>-</code> are valid.
Numeric literal	<code>1</code> , <code>0.5</code> , <code>1.2e-3</code>	Evaluated with <code>f64</code> semantics.
Boolean literal	<code>true</code> , <code>false</code>	Used in conditions and logical expressions.
Parentheses	<code>(a + b) / 2</code>	Use parentheses to override precedence.
Unary operators	<code>-x</code> , <code>!flag</code>	Unary <code>-</code> negates numbers. Unary <code>!</code> negates booleans.
Binary operators	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>^</code> , <code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>&&</code> , <code> </code>	Arithmetic is numeric. Logical operators are boolean.
Local assignment	<code>spread = a - b; spread / 2</code>	Statements run from left to right. The formula must end with a value.
Comments	<code>// line</code> , <code>/* block */</code>	Comments are ignored.

i New formulas should use raw `InstrumentId` values. For backward compatibility, formulas that replace `-` with `_` in component IDs remain accepted.

Operator precedence

The expression engine evaluates operators in the following order, from highest precedence to lowest precedence:

Level	Operators	Notes
Highest	<code>^</code>	Exponentiation. Right associative.

Level	Operators	Notes
	Unary <code>-</code> , unary <code>!</code>	<code>-2 ^ 2</code> evaluates as <code>-(2 ^ 2)</code> .
	<code>*</code> , <code>/</code> , <code>%</code>	Multiplication, division, and modulo.
	<code>+</code> , <code>-</code>	Addition and subtraction.
	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Numeric comparisons.
	<code>==</code> , <code>!=</code>	Equality and inequality. Both sides must have the same type.
Lowest	<code>&&</code> , <code> </code>	Boolean operators.

Assignments are not expression operators. Separate statements with `;`, and make the last statement the value you want the synthetic to produce.

Built-in functions

Function	Signature	Notes
<code>abs</code>	<code>abs(x)</code>	Absolute value.
<code>ceil</code>	<code>ceil(x)</code>	Ceiling.
<code>floor</code>	<code>floor(x)</code>	Floor.
<code>round</code>	<code>round(x)</code>	Round to the nearest integer using Rust <code>f64</code> rules.
<code>min</code>	<code>min(x1, x2, ...)</code>	Accepts one or more numeric arguments.
<code>max</code>	<code>max(x1, x2, ...)</code>	Accepts one or more numeric arguments.
<code>if</code>	<code>if(condition, when_true, when_false)</code>	The condition must be boolean. Both branches match. Only the selected branch evaluates.

Type rules

- Component inputs are numeric.

- Arithmetic operators require numeric operands and return numeric results.
- `<`, `<=`, `>`, `>=` require numeric operands and return boolean results.
- `==` and `!=` accept any matching type (both numeric or both boolean) and return boolean results.
- `&&`, `||`, and unary `!` require boolean operands.
- `&&` and `||` short-circuit. The right-hand side evaluates only when needed.
- Local variables must be assigned before use.
- Local variable names must start with a letter or `_` and then use letters, digits, or `_`.
- The final formula result must be numeric. A formula that ends with an assignment or produces a boolean result is invalid for a synthetic instrument.

Limits

The expression engine enforces the following compile-time limits. Formulas that exceed them produce a clear error at construction time.

Limit	Value	Description
Stack depth	32	Maximum number of intermediate values on the evaluation stack.
Local variables	16	Maximum number of distinct local variable names.

These limits are generous for any realistic pricing formula. A weighted sum of 8 components uses a peak stack depth of 3 and zero locals.

Examples

```
# Simple spread
formula = "BTCUSDT.BINANCE - ETHUSDT.BINANCE"

# Average of two FX pairs
formula = "(AUD/USD.SIM + NZD/USD.SIM) / 2"

# Reuse an intermediate value
formula = "spread = BTCUSDT.BINANCE - ETHUSDT.BINANCE; spread / 2"
```



```
# Conditional output
```

```
formula = "if(BTCUSDT.BINANCE > ETHUSDT.BINANCE, BTCUSDT.BINANCE, ETHUSDT.BINANCE)"
```

Creating a synthetic instrument

Before defining a new synthetic instrument, make sure all component instruments already exist in the cache.

The following example creates a synthetic instrument with an actor or strategy. This synthetic represents a simple spread between Bitcoin and Ethereum spot prices on Binance. It assumes that `BTCUSDT.BINANCE` and `ETHUSDT.BINANCE` already exist in the cache.

```
from nautilus_trader.model.instruments import SyntheticInstrument

btcusdt_binance_id = InstrumentId.from_str("BTCUSDT.BINANCE")
ethusdt_binance_id = InstrumentId.from_str("ETHUSDT.BINANCE")

synthetic = SyntheticInstrument(
    symbol=Symbol("BTC-ETH:BINANCE"),
    price_precision=8,
    components=[
        btcusdt_binance_id,
        ethusdt_binance_id,
    ],
    formula=f"{btcusdt_binance_id} - {ethusdt_binance_id}",
    ts_event=self.clock.timestamp_ns(),
    ts_init=self.clock.timestamp_ns(),
)

self._synthetic_id = synthetic.id
self.add_synthetic(synthetic)
self.subscribe_quote_ticks(self._synthetic_id)
```



 The synthetic `instrument_id` in the example above is `{symbol}.SYNTH`, which produces `BTC-ETH:BINANCE.SYNTH`.

Updating formulas

You can update a synthetic formula at any time.

```
synthetic = self.cache.synthetic(self._synthetic_id)

new_formula = "(BTCUSDT.BINANCE + ETHUSDT.BINANCE) / 2"
synthetic.change_formula(new_formula)

self.update_synthetic(synthetic)
```



Trigger instrument IDs

You can trigger emulated orders from synthetic prices. In the following example, a synthetic instrument releases an emulated order once the synthetic price reaches the trigger condition.

```
order = self.strategy.order_factory.limit(
    instrument_id=ETHUSDT_BINANCE.id,
    order_side=OrderSide.BUY,
    quantity=Quantity.from_str("1.5"),
    price=Price.from_str("30000.00000000"),
    emulation_trigger=TriggerType.DEFAULT,
    trigger_instrument_id=self._synthetic_id,
)

self.strategy.submit_order(order)
```



Performance

Formulas compile once at construction time and evaluate on every incoming component price tick. The expression engine uses a compile-once/eval-many architecture with a zero-allocation f64 stack, so evaluation adds negligible overhead to the tick-processing path.

Measured on Apple M4 Pro, rustc 1.94.1, release profile (opt-level 3):

Evaluation (hot path)

Formula pattern	Time
(A + B) / 2.0	12 ns

Formula pattern	Time
<code>A * 0.4 + B * 0.3 + C * 0.2 + D * 0.1</code>	18 ns
<code>if(A > B, A - B, B - A)</code>	12 ns
<code>spread = A - B; mid = ...; mid + ...</code>	19 ns
<code>max(min(A, B * 20), abs(A - B))</code>	15 ns

Evaluation scaling (weighted sum)

Components	Time
2	14 ns
4	18 ns
8	28 ns

Compilation (cold path)

Formula pattern	Time
Simple average	675 ns
4-input weighted	1.4 us
Conditional	1.0 us
With locals	1.3 us
Hyphenated IDs	755 ns

Error handling

Nautilus validates synthetic instruments at every boundary. Formula compilation rejects unknown symbols, type errors, and capacity overflows. Evaluation rejects wrong input counts and non-finite prices (NaN, Infinity) before they reach the formula.

See the `SyntheticInstrument` [API Reference](#) for input requirements and exceptions.

Related guides

- [Instruments](#) - Instrument definitions and venue-specific instrument types.
- [Data](#) - Market data types that reference instruments.
- [Orders](#) - Orders can use synthetic instrument IDs for emulation triggers.

< Continuous Futures

Previous Page

Value Types >

Next Page